20/1/2023

Digital Electronics

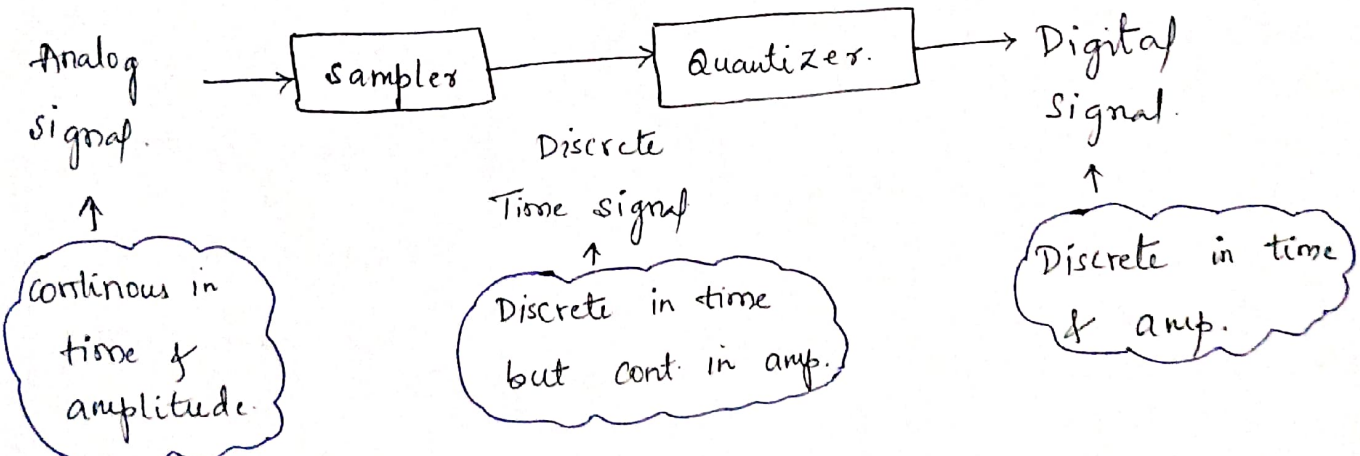
- Digital has been derived from the word "Digit".
- It represent a system with (finite no.) of possibilities.

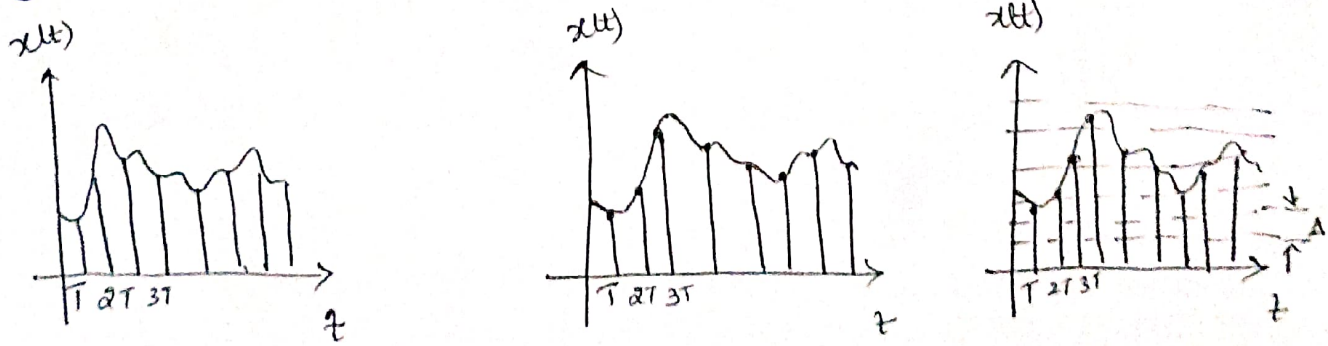
* Why to learn it?

Analog signal are naturally occuring, but to process them, digital circuitry is required due to memory constraint.

Digital processing is definitely better, like in less storage, high speed, better conversion & easy measurement & security of data.

How are Digital signal obtained?





- Due to approximation, there is error introduced called "Quantization Error."
- Here, Δ = step size ∴ Max Error = $\frac{\Delta}{2}$.

- To minimize Quantization Error, we must reduce Δ.
  - ↳ Noise Margin ↓.
  - ↳ as Δ ↓ , no. of level ↑ ; Memory Requirement ↑.

*** Number systems.

* A number system with base 'r', will have r diff-erent digits ranging from '0' to 'r-1'.

* Positional Number system :-

- Position of a digit defines weightage given to a digit.

Ex :- ① $(8357)_{10}$ (positions 3 2 1 0)

$(8 \times 1000) + (3 \times 100) + (5 \times 10) + (7 \times 1)$

$(8 \times 10^3) + (3 \times 10^2) + (5 \times 10^1) + (7 \times 10^0)$

Base		Terminology		Range.
2	⟶	Binary	⟶	0, 1
3	⟶	Ternary	⟶	0, 1, 2.
8	⟶	Octal	⟶	0, - 7.
10	⟶	Decimal	⟶	0 - 9.
16	⟶	Hexadecimal	⟶	0 - 9, A - F.

* 8421 code :- If base is 2, then we can write as: $2^3\ 2^2\ 2^1\ 2^0$ ↙ $8\ 4\ 2\ 1$

This is called as 8421 code.

To avoid confusion i.e 10 is two digit or single. We write (A - F) from (10 - 15)

*. Conversion from One Base system to other :-

① Any to Decimal Number System :- (Multiply)

Ex :- ① $(\overset{2}{5}\overset{1}{2}\overset{0}{7}.\overset{-1}{8})_8 = (?)_{10}$

Soln $(5 \times 8^2) + (2 \times 8^1) + (7 \times 8^0) + (8 \times 8^{-1})$

$\Rightarrow (5 \times 64) + (2 \times 8) + (7 \times 1) + \frac{8}{8}^1$

$\Rightarrow 320 + 16 + 7 + 1$

$\Rightarrow (344)_{10}$

② $(\overset{2}{1}\overset{1}{1}\overset{0}{1}.\overset{-1}{0}\overset{-2}{1}\overset{-3}{1})_2 = (?)_{10}$.

Soln $(1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2}) + (1 \times 2^{-3})$

$\Rightarrow 4 + 2 + 1 + 0 + \frac{1}{4} + \frac{1}{8}$

$\Rightarrow 7 + \frac{1}{4} + \frac{1}{8}$

$\Rightarrow (7.375)_{10}$

② Decimal to Any Number system :-

Ex :- ① $(56.625)_{10} = (?)_2$

Soln Right side multiply. Left side Divide.

2	56
2	28 – 0
2	14 – 0
2	7 – 0
2	3 – 1
	1 – 1

111000

$0.625 \times 2 = 1.25$ (1)
$0.25 \times 2 = 0.5$ (0)
$0.5 \times 2 = 1.0$ (1)

101

$(56.625)_{10} = (111000.101)_2$.

② $(712)_{10} = (?)_8$

Soln

$8 \mid 712$

$8 \mid 89 - 0$

$8 \mid 11 - 1$

$1 - 3$

$(712)_{10} = (1310)_8$

③ Binary to Octal Number System :-

Ex :- ① $(1001101.1001)_2$

Soln

001 001 101 . 100 100 — 4 2 1 :: code

$(1\ 1\ 5 \cdot 4\ 4)_8$ //

④ Octal to Binary Number System :-

Ex :- $(115.44)_8 = (?)_2$

$(001\ 001\ 101 . 100\ 100)_2$ //

⑤ Binary to Hexadecimal Number System :-

Ex :- ① $(110100111.110011)_2 = (?)_{16}$

Soln

0001 1010 0111 . 1100 1100

$(1\ A\ 7 \cdot C\ C)_{16}$ //

⑥ Hexadecimal to Binary Number system :-

Ex :- $(A11.BCF)_{16} = (?)_2$

Sol $(1010\ 0001\ 0001\ 1011\ 1100\ 1111)_2$

1Q. Given $(135)_x + (144)_x = (323)_x$.

What is the value of x ?

Soln

$$x^2 + 3x + 5 + x^2 + 4x + 4 = 3x^2 + 2x + 3$$

$$2x^2 + 7x + 9 = 3x^2 + 2x + 3$$

$$3x^2 - 2x^2 + 2x - 7x + 3 - 9 = 0$$

$$x^2 - 5x - 6 = 0.$$

$$(x-6)(x+1) = 0.$$

$$\therefore \boxed{x = 6}$$

2Q). Consider the equation $(43)_x = (y3)_8$, where x & y are unknown. The no of possible solutions is :-

Soln

$$4x + 3 = 8y + (3 * 8^0)$$

$$4x + 3 = 8y + 3$$

$$4x = 8y$$

$$\therefore \boxed{x = 2y}$$

Here, base - x is given value from the 4, value $\boxed{x > 4}$ or $x \geq 5$

& y value lies b/w $y: 0 \rightarrow 7$.

$\therefore$ $x = 6$, $y = 3$; $x = 10$, $y = 5$; $x = 14$, $y = 7$.

$x = 8$, $y = 4$; $x = 12$, $y = 6$.

$\therefore$ No of possible solutions will be only (5)

Q. If $(73)_x = (54)_y$, possible values of x & y.

a) 8 & 16 c) 9 & 13

b) 10 & 12 d) 8 & 11.

Soln

$7x+3 = 5y+4$

$\therefore 7x - 5y = 1$

$7(8) - 5(11) = 1$

$1 = 1$

(d) 8 & 11

//

Q) If $(11X1Y)_8 = (12C9)_{16}$, then the values of X & Y are

a) 5 & 1 b) 5 & 7 c) 7 & 5 d) 1 & 5 ~~e) 3 & 1~~

Soln

1 2 C 9

0001 0010 1100 1001

$(1\ 1\ 3\ 1\ 1) = (11X1Y)$

~~e)~~ $X = 3, Y = 1$ //

* Data Representation :-

① Unsigned Magnitude Representation :-

- Here we represent only +ve Numbers.
- Range : With N Bits, we can represent $\boxed{0 \text{ to } 2^N - 1}$.

If N = 3, then we can represent

000 = 0

to

111 ⇒ 7 i.e (0 to $2^N - 1$)

② signed Magnitude Representation :-

- Both +ve & -ve no can be represent.
- M.S.B is dedicated to represent the sign.
- Range :- $\boxed{-(2^{N-1}-1) \text{ to } (2^{N-1}-1)}$

Unsigned :- If $N=8$; $= 2^8 - 1$

$= 255 \quad (0 \to 255) \Rightarrow (256)$

signed :- If $N=8$; $= 2^{N-1} - 1$

$\Rightarrow 2^{8-1} - 1 \Rightarrow 2^7 - 1 \Rightarrow 127$

$(-127 \text{ to } 127) \Longrightarrow (255)$.

Here signed Magnitude Representation less is ~~less~~ range because 0 can be represent in 2 form:

$\{+0 = 0\ 0000000$
$-0 = 1\ 0000000\}$.

③ 1's Complement Representation :-

Soln (i) If it is a positive Number then simply write its binary representation.

$(6)_{10} = (0\ 110)_2$
↑
sign

Range will be same as signed Magnitude. *** (255)

(ii) If it is a -ve No then write compliment of binary No system.

1's. compliment ⇒ $(0\ 110)$
↓
$(1\ 001)_2$
↑
sign

+0 : 0 0000 — 5 bit → ① sign, ④ Mag
-0 : 1 1111
↓
(1's comp).

$\therefore (-6)_{10} \Rightarrow (1001)_2$. //-

(11) 2's Complement Representation :-

2's complement = 1's complement + 1

Range of Numbers :- $(2^{N-1}-1)$ & $(-2^{(N-1)})$

$$\therefore \boxed{-2^{N-1} \text{ to } 2^{N-1}-1}$$

+0 : 0 00 00 : 5 bit ⟷ 1 bit sign, 4 bit Mag.
-0 : 1 11 11 (1's Comp).

```
  1 1 1 1
  1 1 1 1 1
          1
 (1) 0 0 0 0 0   ← 2's comp.
  ↑
carry.
```

∴ +0 representation is same as -0.

∴ -no representation has been increased.

8 bits : -2^{8-1} to $2^{8-1}-1$

(-128 to 128) *** 256 Total Bits.

If the no is +ve then write only binary representation

```
          8 4 2 1
+5 :-     0 0 1 0 1
          ↑
         +ve.
```

for -ve no write binary rep & do 2's comp. (-5)

(+5) :- 0 0 1 0 1
1's compliment :- 1 1 0 1 0
2's compliment :- 1
1 1 0 1 1 → (-5)

**** Data Representation :-

Representation	Range
Unsigned Magnitude Representation	$(0 \text{ to } 2^N - 1)$
Signed Magnitude Representation	$-(2^{N-1} - 1)$ to $(2^{N-1} - 1)$
signed 1's Complement Representation	$-(2^{N-1} - 1)$ to $(2^{N-1} - 1)$
signed 2's complement Representation.	$-(2^{N-1})$ to $(2^{N-1} - 1)$.

* Binary Codes :- *** Error Detection Code.

① BCD Code :- Binary Coded Decimal.

- Every decimal Digit (0-9) is represented as a comb^{n} of 4 bits.
- Total no of comb^{n} 4 bits = 16. → Used = 10, → Unused = 6.

Decimal No.	BCD code.
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001.

$(396)_{10} \Rightarrow (0011\ 1001\ 0110)_{BCD}$

Represent Each digit of decimal no by 4 bit comb^{n}.

- Binary code ② Gray Code :- {Any 1 Bit changes} is only ①

Binary :- 0 0 1 1 → Gray :- 1 1 0 0

to to

Gray 0 0 1 0 Binary :- 1 0 0 0

non-weighted codes.

③ Excess - 3 Code :- BCD + 3 (0011). weighted code.

(2421), (5421), (Excess -code) are called Reflective code.

self complimentary is other other.

④ Sequential code :- succeeding code is one binary no greater than its preceding code

Ex:- BCD, Excess - code.

⑤ Alphanumeric Code :-

- Binary digit can only represent two symbols 0 or 1.
- But this is not enough for communications b/w two computer.
- To overcome this we have Alphanumeric codes which has 26 alphabet & atleast 10 digit & ∴ total (36) data are present.

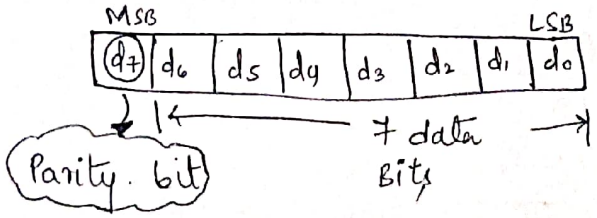
① American standard code for Information Technology (ASCII). — 7-Bit

② Extended Binary Coded Decimal Interchange Code (EBCDIC). — 8-Bit

③ Five Bit Baudot code.

⑥ Error Detecting & Correcting code :-

Ex:- Parity check.



Even & odd parity.

- If sender send even parity & receiver gets odd parity then it is called Error detecting.

Ex:- Hamming code is also used.

***. It is used only (1)-Bit changes.

- If more than 1-bit changes, then this code cannot be used.

Binary Arithmetic.

①. Binary Addition :-

8-bit = byte; 4-bit = Nibble.

A	+	B	Sum	Carry
0		0	0	0
0		1	1	0
1		0	1	0
1		1	0	1

i> If the lower/higher-order (4-bit) nibble of the result is "> 9" (or) Auxiliary carry = 1, then add "6" to the lower nibble of the result.

Ex :- 65 + 29.

```
65 :- 0110  0101
29 :- 0010  1001
      1000  1110
```

14 > 9 :- Add 6 to lower nibble.

```
65 :  0110  0101
29 :  0010  1001
      ----------
            1110
          + 0110 (6)
      ----------
      1001  0100
```

(9) (4)

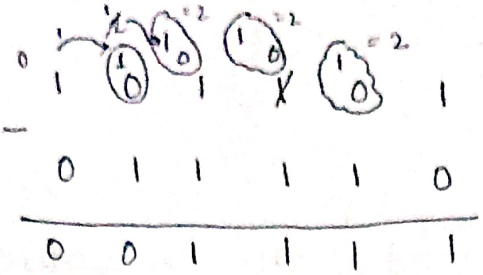
②. Binary Substraction :-

A - B	Substract / Diff	Borrow
0 0	0	0
0 1	(1)	(1)
1 0	1	0
1 1	0	0

→ 1 Borrow :- 0 becom (10) i.e 2.

2 - 1 = 1

* Binary Subtraction Ex :-



```
  0 1 1 1 1 0
-------------
  0 0 1 1 1 1
```

* Subtraction by 1's complement :-

(A − B) ⟶ A : minuend.
B : subtrahend.

{A + 1's complimnt of B}.

- If the result is having carry = 1, Then add LSB of result.
- " " " " " no carry, take 1's compl. of result.

Ex :- ① 10101 − 00101

Soln

1's compl of 00101 ⇒ 11010

```
  10101
+ 11010
-------
C=1 01111
    ↓ 1
-------
  10000  = (16)10
```

② $10^{21}101 - 1^{30}1110$

Soln 1's compl. of 11110 ⇒ 00001

```
  10101
+ 00001
-------
  10110
```

C = 0

1's compl. of result ⇒ $(01001)_2$ ⇒ 9

Add -ve to the answer

$(-9)_{10}$

* Substraction using 2's compliment :-

A - B

(A + 2's compl. of B)

- If the result is having carry, then we discard it.
- If the result is having no carry, then take 2's compl. of result.

Ex :- $10101 - 00111$ (21 − 7 = 14)

Sol^n 2's compl. of 00111 ⇒ 11000 + 1 = 11001

$$\begin{array}{r} 1 \\ 10101 \\ +\ 11001 \\ \hline 01110 \end{array}$$

C = 1 (discard)

8 4 2 1 ⇒ 14

Discard carry. //

(3). Binary Multiplication :-

A x B	Multiplication
0 0	0
0 1	0
1 0	0
1 1	1

(4) Overflow :-

- When there is insufficient bits in binary no.
- Due to overflow sign bit changes which gives wrong result.

Logic Gates.

- Logic gates are basic building block of a digit ckt which is used to make logical decisions.

0 = Logic '0' 1 = Logic '1'

- In electrical terms, logic 1 & logic 0 are represented by 2 v/g levels.

Unipolar coding :-
one level = 5v.
2nd level = 0v.

Bipolar coding :-
one level = 5v
2nd level = -5v.

BJT = 5v
MOSFET = 3.3v.

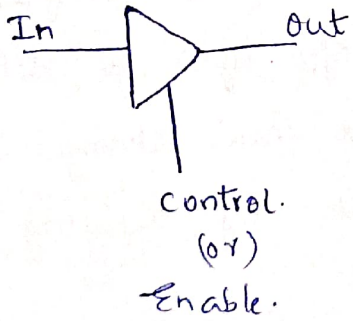
Positive logic :-
Logic '0' = 0v
Logic '1' = 5v

Negative logic :-
opposite. Logic '1' = 0v.
Logic '0' = 5v.

① Buffer :-

A	X
0	0
1	1

② Tri - State Buffer :-

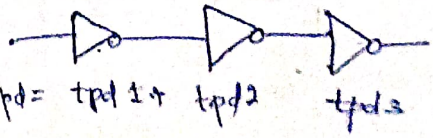


3 states of o/p :- Logic 0, Logic 1, (Hi - z)
↓
Open ckt.

Control	In	out
0	0/1	Hi-Z
1	0	0
1	1	1

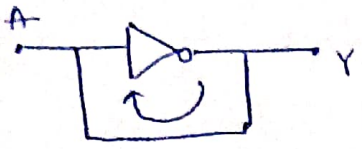
③ NOT Gate :-

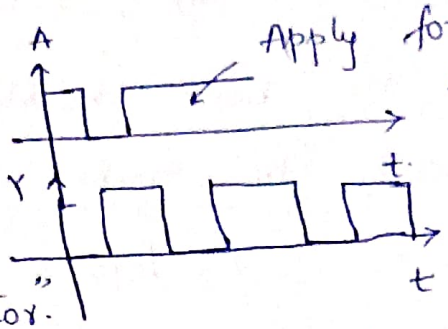
A	B
0	1
1	0



∴ Total tpd = n*tpd.

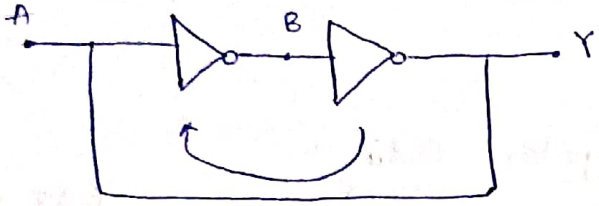
* Inverters with Feedback :-

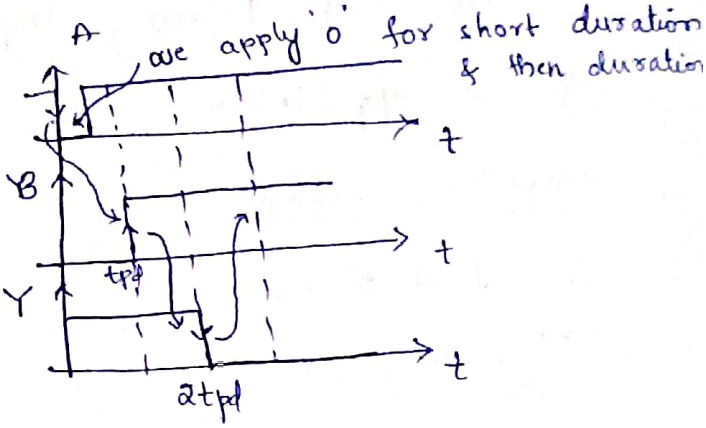




→ Square wave Generator.

* "Astable Multivibrator".





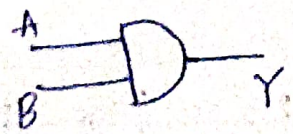
* When we apply '0' as trigger, the o/p becomes 0 at 2tpd & stays at '0'.

* Hence it is called as "Bistable Multivibrator".

- Odd no of inverters in feedback loop.
  - ↳ Astable Multivibrator.
  - ↳ Generate square wave.
  - ↳ Period = 2n·tpd
  - ↳ Duty cycle = 50%

- Even no of inverters in feedback loop.
  - ↳ Bistable MV (or) Memory Element.
  - ↳ Stores the trigger value at o/p after a delay of n·tpd
  - ↳ we can change stored value by applying trigger.

④ AND Gate :-



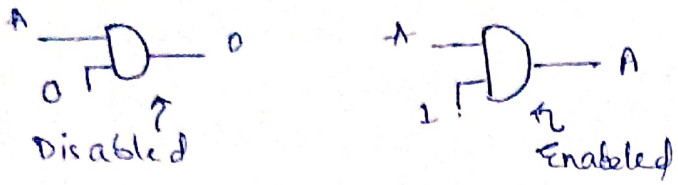
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

* Properties of AND Gate :-

① Commutative Property :- $A \cdot B = B \cdot A$.

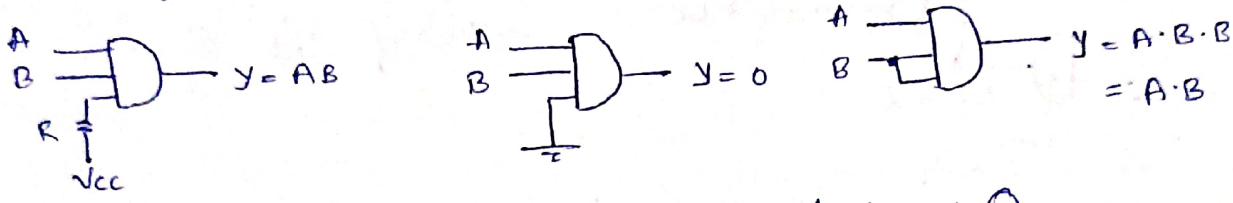
② Associative Property :- $(A \cdot B) \cdot C = A \cdot (B \cdot C)$

* Enabled & Disabled AND Gate :-



*** Imp

* Floating I/p in AND Gate :-

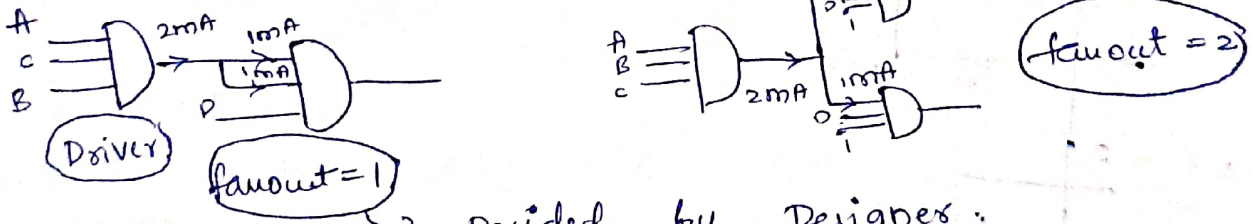


TTL :- Transistor - Transistor Logic ⇒ ①.

ECL :- Emitter Coupled Logic ⇒ ⓪. ← fastest logic family.

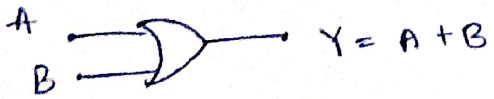
Fanout :- No of logic gates that can be connected at o/p of one logic Gate (or).

How many no of times that the driver gate can run the upcoming gate



Decided by Designer.

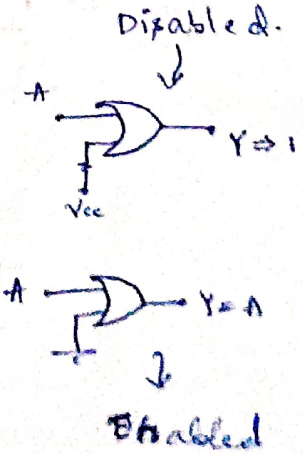
⑤. OR Gate :-



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Property :- ⓪. $A + B = B + A$.

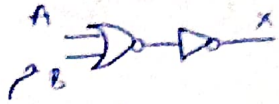
① $(A + B) + C = A + (B + C)$.



⑥. NOR Gate :-

Truth Table :-

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0



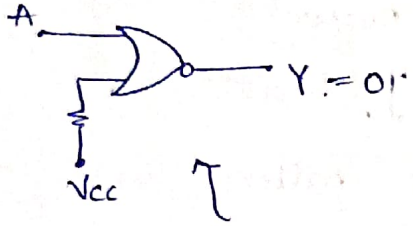
• Practically "OR" Gate has been derived from (NOR + NOT) Gate.

• Properties :-

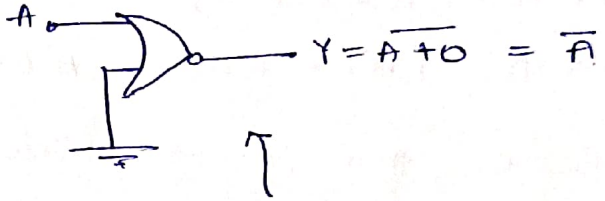
① Commutative Property :- $\overline{A+B} = \overline{B+A}$

② Associative Property :- $\overline{(\overline{A+B})+C} \neq \overline{A+(\overline{B+C})}$

• Enabled & Disabled :-

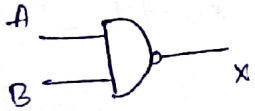


Disabled



Enabled.

⑦. NAND Gate :-



Truth Table :-

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

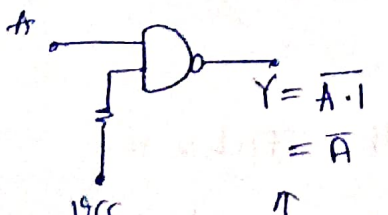
• Practically AND Gate has been derived from (NAND + NOT) Gate

• Properties :-

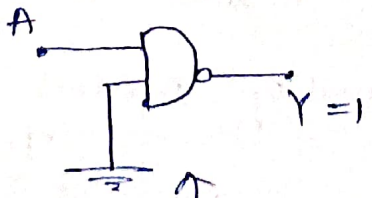
① Commutative Property :- $\overline{A \cdot B} = \overline{B \cdot A}$.

② Associative Property :- $\overline{(\overline{A \cdot B}) C} \neq \overline{A (\overline{B \cdot C})}$.

• Enabled & Disabled :-



Enabled



Disabled.

⑧. XOR Gate :- (Exclusive OR Gate) : Inequality Detector.

$Y = A \oplus B$

$= A\bar{B} + \bar{A}B.$

Truth Table :-

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

· Properties :-

① Commutative Property :- $A \oplus B = B \oplus A$.

② Associative Property :- $(A \oplus B) \oplus C = A \oplus (B \oplus C)$.

③. $A \oplus A = 0$.

④. $A \oplus \bar{A} = 1$.

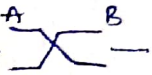
⑤. $A \oplus 0 = A \quad [A \cdot \bar{0} + \bar{A} \cdot 0] = A \cdot 1 = A$. {Buffer}

⑥. $A \oplus 1 = \bar{A} \quad [A \cdot \bar{1} + \bar{A} \cdot 0 = A \cdot 0 + \bar{A} = \bar{A}]$. (Inverter)

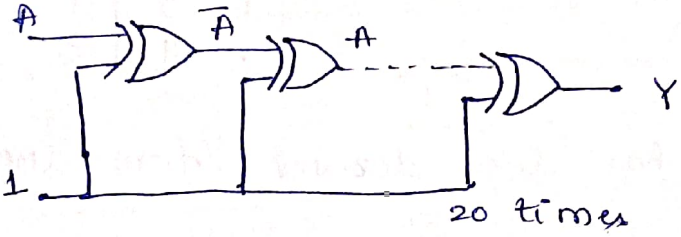
⑦ $A \oplus B = C$ then it will follow cyclic pattern as:

$B \oplus C = A$

$A \oplus C = B$.

XOR Gate is formed by using a way switch 

Q>.



Sol^n $A \oplus 1 \Rightarrow A \cdot 0 + \bar{A} \cdot 1 \Rightarrow \bar{A}$

$\bar{A} \oplus 1 \Rightarrow \bar{A} \cdot 0 + \bar{A} \cdot 1 \Rightarrow A$

At 2nd stage value is A, (Buffer) so at 20 stage value is 10 times (Buffer). $\Rightarrow$ (A). //

⑨. XNOR Gate :- (equality Detector)

$Y = \overline{A \oplus B} = A \odot B$

$= AB + \bar{A}\bar{B}.$

Truth Table :-

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

· Properties :-

(1). Commutative Property :- $A \odot B = B \odot A$.

(2). Associative Property :- $(A \odot B) \odot C = A \odot (B \odot C)$

(3). $A \odot A = 1$

(4) $A \odot \bar{A} = 0$.

(5). $A \odot 1 \Rightarrow A$ $\quad A(1) + \bar{A}(0) \Rightarrow A$

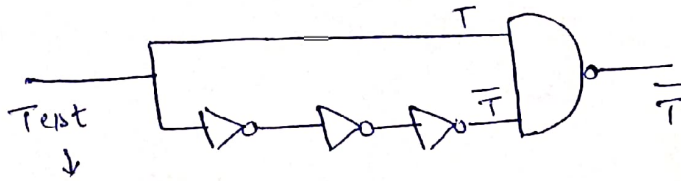
(6) $A \odot 0 \Rightarrow \bar{A}$ $\quad \bar{\bar{A}}(0) + \bar{A}(\bar{0}) = \bar{A}(1) = \bar{A}$.

(7). $A \odot B = C$

then, $B \odot C = A$

$C \odot A = B$.

Qx.

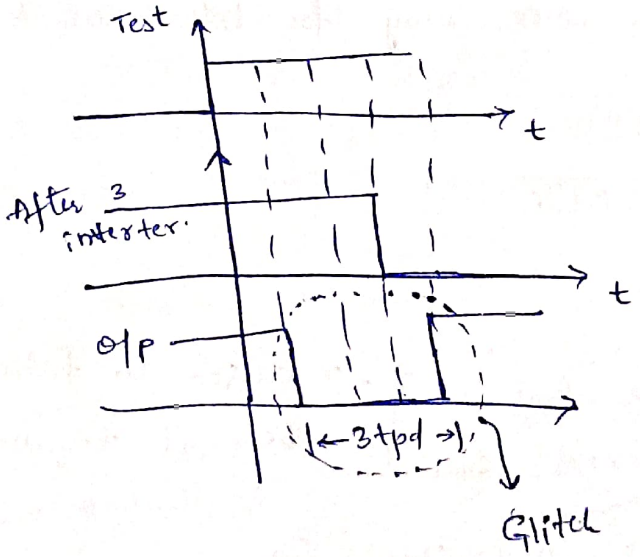


$\overline{T \cdot \bar{T}} \Rightarrow \overline{1 \cdot 0} \Rightarrow \bar{0} = 1$.

starting Low, is switched to high & maintained at high.

The o/p = ? (Assume Non-zero tpd).

Soln



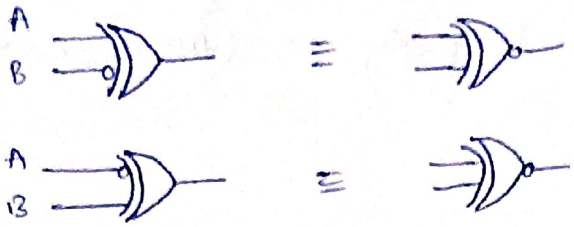
(D) pulse from high to Low to high.

* Relation b/w XOR & XNOR Gate:-

① 1 I/P Bubbling Effect :-

$\bar{A} \oplus B = A \odot B$

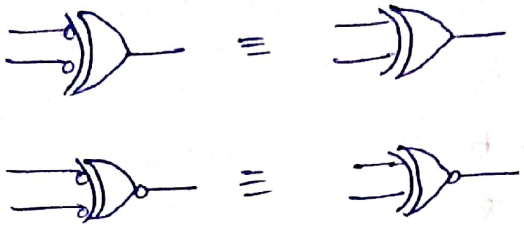
$A \oplus \bar{B} = A \odot B$



② 2 I/P Bubbling Effect :-

$\bar{A} \oplus \bar{B} = A \oplus B$

$\bar{A} \odot \bar{B} = A \odot B$

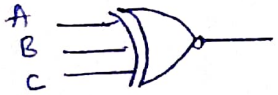


$A \odot B = \overline{A \oplus B}$

$A \oplus B = \overline{A \odot B}$

- XOR Gate o/p = 1, when odd no of (1) i/p are present.
- XNOR Gate o/p = 1, when even no of (0) i/p are present.

(3) i/p behave in same way for both XOR & XNOR.



$Y = \overline{A \oplus B \oplus C}$

$Y = \overline{A \odot B \odot C}$

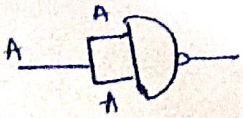
* Universal Gates :-

"NAND Gate" & "NOR Gate" are called Universal Gate. → Easier to fabricate beoz it require less no of transistors.

& also Inhibition & Implication Gate.

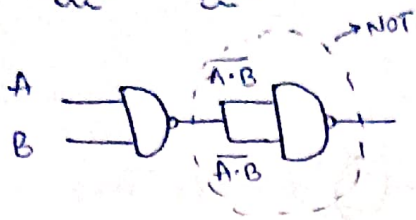
* Implementation :-

① NOT :-



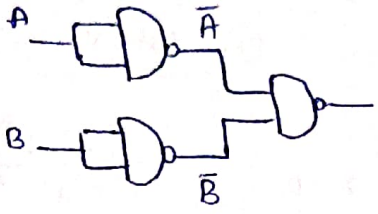
$Y = \overline{A \cdot A} \Rightarrow \bar{A}$

② AND Gate :-



$Y = \overline{\overline{A \cdot B}} = A \cdot B$

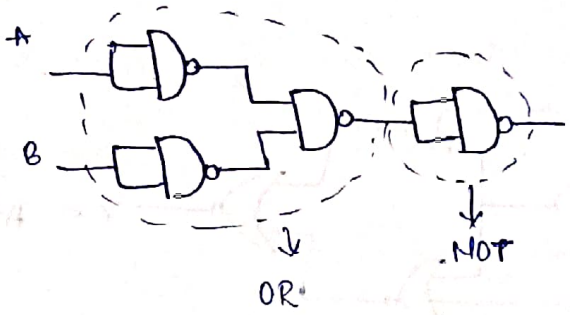
③ OR Gate :-



$\overline{\overline{A} \cdot \overline{B}} \Rightarrow \overline{\overline{A}} + \overline{\overline{B}}$

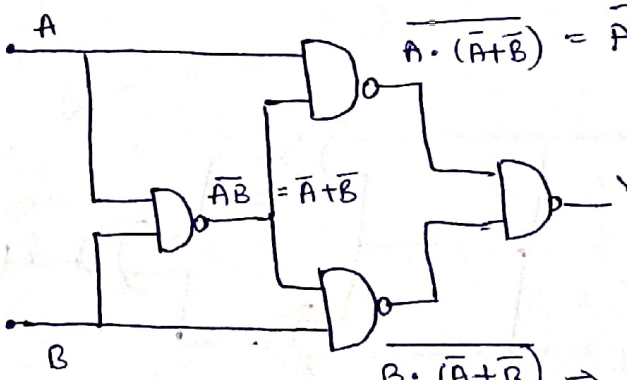
$\Rightarrow A + B$

④ NOR Gate :-



= NOR Gate

⑤ XOR Gate :-



$\overline{A \cdot (\overline{A}+B)} = \overline{\overline{A} \cdot B} = \overline{A} + B$

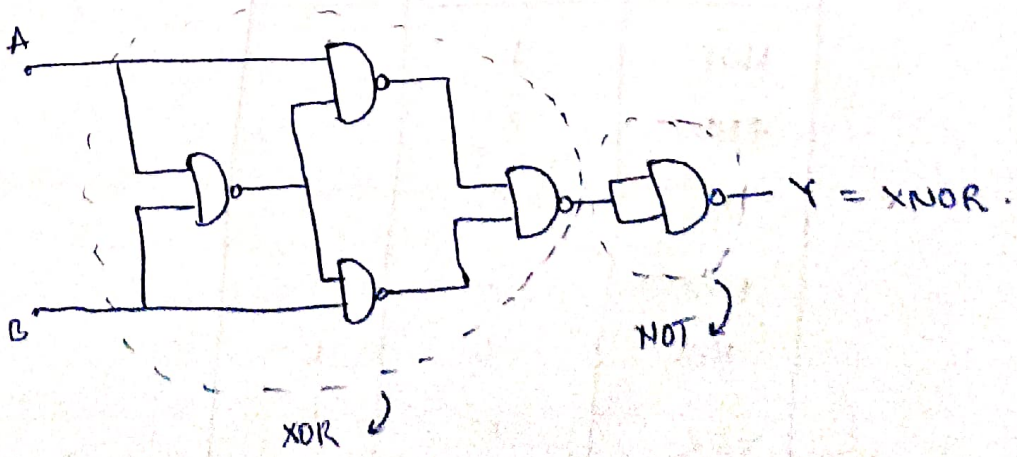
$Y = \overline{(\overline{A}+B) \cdot (A+\overline{B})}$

$\Rightarrow \overline{\overline{A}B} + \overline{A\overline{B}}$

$\Rightarrow A \cdot \overline{B} + \overline{A} \cdot B = XOR$

$\overline{B \cdot (\overline{A}+B)} \Rightarrow \overline{\overline{A}B} \Rightarrow A + \overline{B}$

⑥ XNOR Gate :-

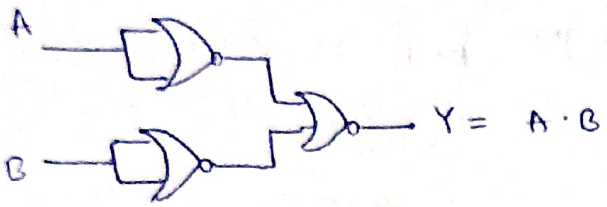


Implementation of NOR Gate:-

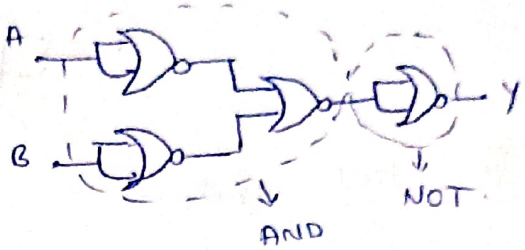
① NOT Gate:-

② OR Gate:-

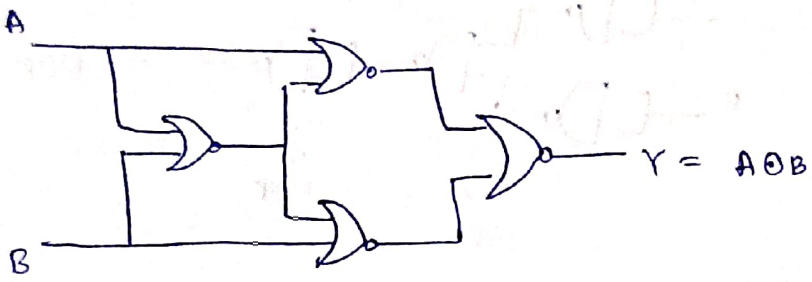
③ AND Gate:-



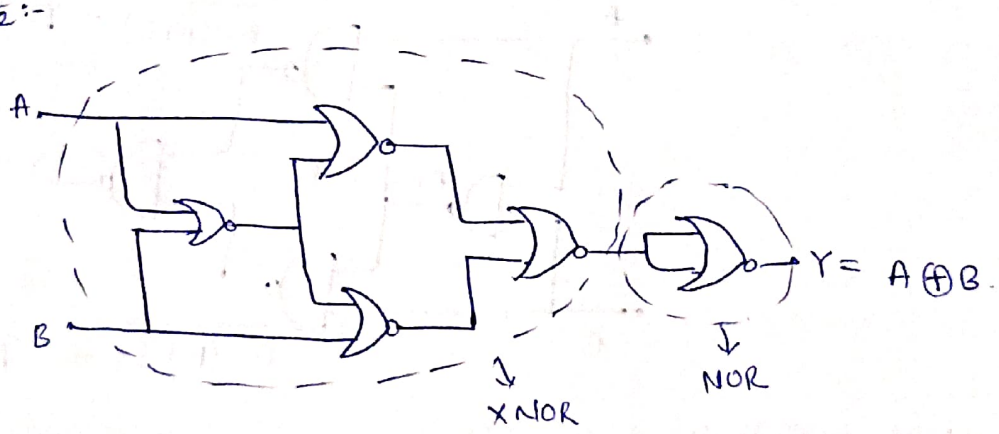
④ NAND Gate:-



⑤ XNOR Gate:-

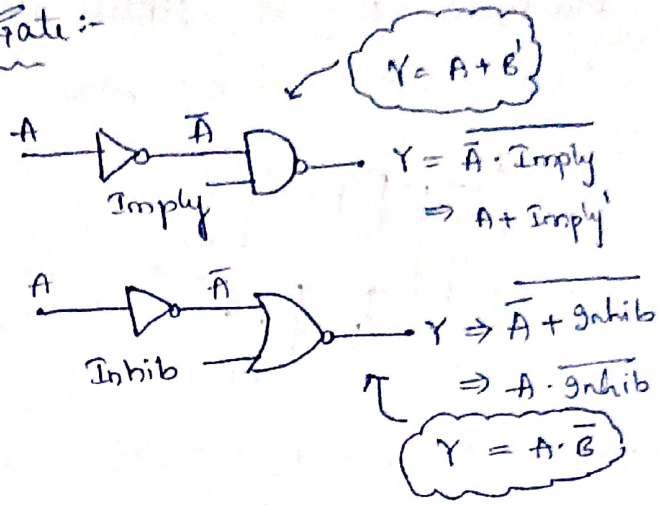


⑥ XOR Gate:-



Gate	NAND	NOR
NOT	1	1
AND	2	3
OR	3	2
NAND	1	4
NOR	4	1
XOR	4	5
XNOR	5	4

* Implication & Inhibition Gate :-



* Imply = 1, $Y = \overline{\bar{A} \cdot 1} = A$

= 0, $Y = \overline{\bar{A} \cdot 0} = 1$.

If Imply = 1 ; o/p = A

i.e if Imply is true, o/p follows i/p.

* Inhib = 1, $Y = \overline{\bar{A} + 1} = \overline{1} = 0$.

= 0, $Y = \overline{\bar{A} + 0} = \bar{\bar{A}} = A$.

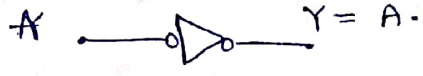
Imp. ** Both this Gate also called Universal Gate

If Inhib = True, i/p is inhibited from reaching o/p

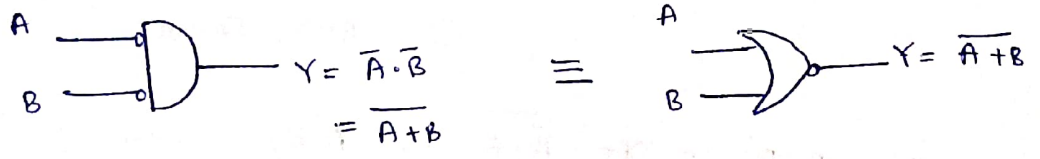
* Bubbled Input Gate :-

(1) Buffer :-

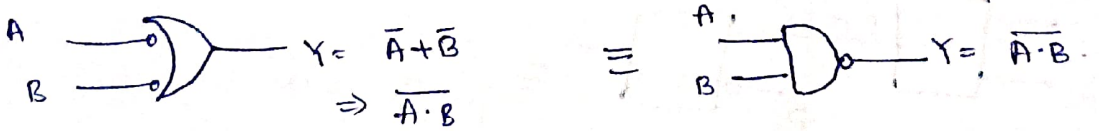
(2) Inverter :-



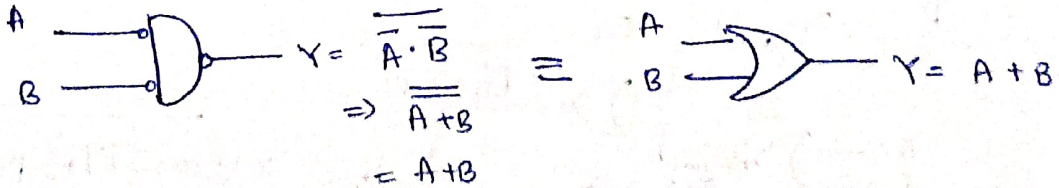
(3). AND :-



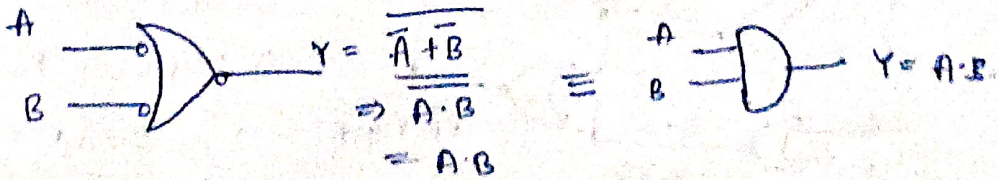
(4). OR :-

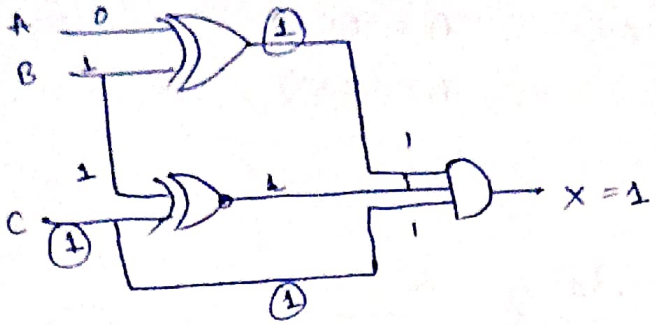


(5). NAND :-



(6). NOR :-



Q) 

A, B, C = ?

∴ A=0, B=1, C=1

Boolean Algebra.

- Boolean Laws :-

① AND Law :-

1. $A \cdot 0 = 0$
2. $A \cdot 1 = A$
3. $A \cdot A = A$
4. $A \cdot \bar{A} = 0$

② OR Law :-

1. $A + 0 = A$
2. $A + 1 = 1$
3. $A + A = A$
4. $A + \bar{A} = 1$

③ Commutative Law :-

$A + B = B + A$

$A \cdot B = B \cdot A$

④ Associative Law :-

$A \cdot (B \cdot C) = B \cdot (C \cdot A)$

⑤ Distributive Law :-

$A + (BC) = (A+B) \cdot (A+C)$

⑥ De-Morgan's Theorem :-

$$\overline{AB} = \bar{A} + \bar{B}$$

$$\overline{A+B} = \bar{A} \cdot \bar{B}$$

1Q) Solve $A(\bar{A}+C)(\bar{A}B+C)(\bar{A}BC+C')$

Soln

$A\bar{A}(1+C)(B+C)(BC+C')$ ($A\bar{A} = 0$)

$\Rightarrow 0$

* Duality :-

Ex :- $A + BC$

$A \cdot (B+C)$

Interchange :- $1 \leftrightarrow 0$.

AND $\leftrightarrow$ OR.

(7). Consensus theorem / Redundancy theorem :-

$BC + AB + A'C \Rightarrow AB + A'C$.

BC → Redundant term.

- Do not remove the terms that contain variable in one & its comp. in others.

$(A+B)(A'+C)\cdot(B+C) = (A+B)\cdot(A'+C)$.

* Conditions for applying Redundancy theorem :-

(1). Each variable is repeated twice.

(2). One variable must present in both comp. & uncomp. form.

- $A + A'B = (A+B)$ ⟸ .

  A' → Redundant term/literal. (RWL)

- $A(A'+B) = A \cdot B$.

2Q). Simplify $Y = A\bar{B} + (\bar{A}+B)\cdot C$.

Soln

$Y \Rightarrow A\bar{B} + \overline{A\cdot\bar{B}}\cdot C$

Let $X = A\bar{B}$

$Y = X + X'C$. $(A + \bar{A}B = A+B)$

$\therefore \boxed{Y = A\bar{B} + C}$

• Minterms :-

• A **minterm** of n variables is a product of variables in which each appears exactly once in true or complemented form. Represented by (m).

A	B	C	m		M	
0	0	0	$\bar{A}\bar{B}\bar{C}$	m_0	$A+B+C$	M_0
0	0	1	$\bar{A}\bar{B}C$	m_1	$A+B+\bar{C}$	M_1
0	1	0	$\bar{A}\bar{B}\bar{C}$	m_2	$A+\bar{B}+C$	M_2
0	1	1	$\bar{A}BC$	m_3	$A+\bar{B}+\bar{C}$	M_3
1	0	0	$A\bar{B}\bar{C}$	m_4	$\bar{A}+B+C$	M_4
1	0	1	$A\bar{B}C$	m_5	$\bar{A}+B+\bar{C}$	M_5
1	1	0	$AB\bar{C}$	m_6	$\bar{A}+\bar{B}+C$	M_6
1	1	1	ABC	m_7.	$\bar{A}+\bar{B}+\bar{C}$	M_7

• A **maxterm** of n variables is a sum of the variables in which each appears exactly once in true or complemented form. Represented by (M).

① Ex:- $F = \overset{111}{ABC} + \overset{110}{ABC'} + \overset{101}{A\bar{B}C} + \overset{011}{\bar{A}BC}$

(7, 6, 5, 3)

Minterms :- $F = m_3 + m_5 + m_6 + m_7$.

$F = \Sigma m(3, 5, 6, 7)$.

② $F = (\overset{0}{A}+\overset{0}{B}+\overset{0}{C}) \cdot (\overset{0}{A}+\overset{0}{B}+\overset{1}{\bar{C}}) \cdot (\overset{0}{A}+\overset{1}{\bar{B}}+\overset{0}{C}) \cdot (\overset{1}{\bar{A}}+\overset{0}{B}+\overset{0}{C})$

Maxterm: $F \Rightarrow M_0 \cdot M_1 \cdot M_2 \cdot M_4$

$\Rightarrow \pi M(0, 1, 2, 4)$

• Sum of All Minterms ⇒ 1.

• Product of All Maxterms ⇒ 0.

- $m_i = \overline{M_i}$; $M_i = \overline{m_i}$ (Minterm & Maxterm are complement of each other)
- $m_i = M^D_{2^n-1-i}$; $M_i = m^D_{2^n-1-i}$

(First minterm is the dual of last maxterm)

Ex :- $n=3$;

$m_i = M^D_{7-i}$

$m_0 = M^D_7$; $m_1 = M^D_6$ & so on..

Dual :- Replace $0 \leftrightarrow 1$ change AND $\leftrightarrow$ OR. Keep variable as it is.

V/s

Complement :- Replace $0 \leftrightarrow 1$ change AND $\leftrightarrow$ OR complement each variable

*. Literal :-

Each variable in comp & uncomp form in a logic fn.

$F = \overline{A} + BC + \overline{B}C\overline{D} + A\overline{B}\,\overline{C}\,\overline{D}$

(① ② ③ ④)

∴ Literals = 1 + 2 + 3 + 4 ⇒ 10 literals.

- Sum of Products (SOP) → Ex :- $A + BC + B'CD$
- Product of Sum (POS) → Ex :- $(A+B).(A+C).(A+D)$
- Canonical SOP → Ex :- $p'q'r + pq'r + pqr' + pqr$

011 101 110 111

3 5 6 7

∴ $(f = m_3 + m_5 + m_6 + m_7)$

∴ $f = \Sigma m(3, 5, 6, 7)$.

- Canonical POS :- Ex :- $M_0 \cdot M_1 \cdot M_2 \cdot M_4$

Ex :- $\Pi M(0, 1, 2, 4)$

- Standard SOP & POS forms :- Main advantage is that the no of i/p applied to logic gate can be minimized.

* Converting Standard SOP to Canonical SOP:

Ex: $f(A,B,C) = A + BC$

$\Rightarrow A \cdot 1 \cdot 1 + 1 \cdot B \cdot C$

$\Rightarrow A \cdot (B+\bar{B})(C+\bar{C}) + (A+\bar{A}) \cdot B \cdot C$

$\Rightarrow A(BC + B\bar{C} + \bar{B}C + \bar{B}\bar{C}) + ABC + \bar{A}BC$

$\Rightarrow ABC + AB\bar{C} + A\bar{B}C + A\bar{B}\bar{C} + ABC + \bar{A}BC$

$\Rightarrow \overset{1\,1\,1}{ABC} + \overset{1\,1\,0}{AB\bar{C}} + \overset{1\,0\,1}{A\bar{B}C} + \overset{1\,0\,0}{A\bar{B}\bar{C}} + \overset{0\,1\,1}{\bar{A}BC}$

$= \Sigma m(7, 6, 5, 4, 2)$

* Converting standard POS to Canonical POS:-

Ex: $F(A,B,C) = (A+B) \cdot (B+C)$

$\Rightarrow (A+B+0) \cdot (0+B+C)$

$\Rightarrow (A+B+C\cdot\bar{C}) \cdot (A\cdot\bar{A}+B+C)$

$\Rightarrow (\overset{0}{A}+\overset{0}{B}+\overset{0}{C}) \cdot (\overset{0}{A}+\overset{0}{B}+\overset{1}{\bar{C}}) \cdot (\overset{0}{A}+\overset{0}{B}+\overset{0}{C}) \cdot (\overset{1}{\bar{A}}+\overset{0}{B}+\overset{0}{C})$

$\Rightarrow \pi M(0, 1, 4)$

* No of Logical Expressions:-

n variables :: 2^n minterms / Maxterms.

:. No of logical expression using n variable

$= \boxed{2^{2^n}}$.

Q: $X(P, Q, R) = \pi(0,5)$. What are these gates?

Sol:

$\Rightarrow (A+B+C) \cdot (\overset{1}{\bar{A}}+\overset{0}{B}+\overset{1}{\bar{C}})$

$\Rightarrow (P+Q+R) \cdot (\bar{P}+Q+\bar{R})$

$\Rightarrow PQ + P\bar{R} + \bar{P}Q + Q + Q\bar{R} + \bar{P}R + QR$

$\Rightarrow Q(P + \bar{P} + 1 + \bar{R} + R) + P\bar{R} + \bar{P}R$

$\Rightarrow Q + P \oplus R$

2). Given f_1, f_3 & f in canonical SOP for Ckt.

$f_1 = \Sigma m(4,5,6,7,8)$

$f_3 = \Sigma m(1,6,15)$

$f = \Sigma m(1,6,8,15)$

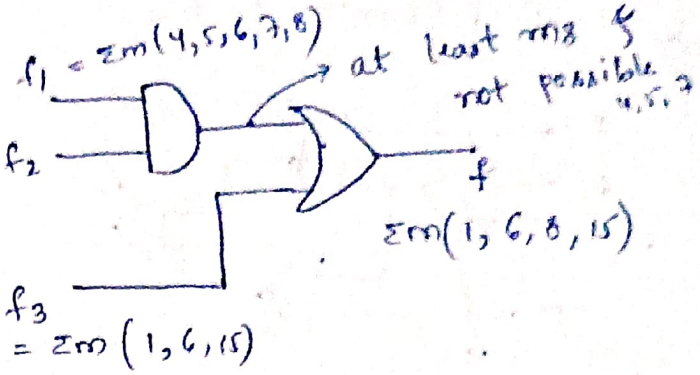
Then (f_2) = ?

(a). $\Sigma m(4,6)$

(b). $\Sigma m(4,8)$

(c) $\Sigma m(6,8)$

(d). $\Sigma m(4,6,8)$



*** In POS:-

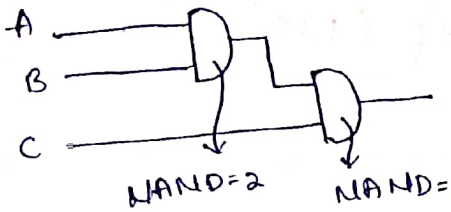
AND Gate :- Union

OR Gate :- Intersection.

. Minimum Number of Gates :-

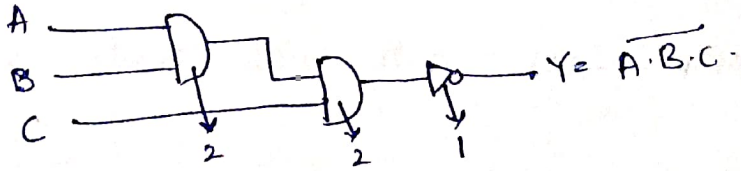
• If we have just a single term which cannot be minimized, then we directly implement.

(1). $Y \Rightarrow A \cdot B \cdot C$



∴ Min no of NAND Gate = 2 + 2 = (4).

(2). $Y = \overline{A \cdot B \cdot C}$



Min. Gate = 5

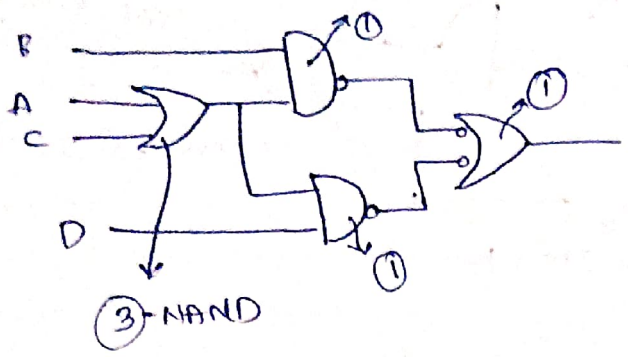
(*). Case-(2) :-

When a standard SOP fn is given.

- Min. logic expression, implement logic expression using NOT, AND, OR Gate
- Replace all gate by NAND Gate.

Eg: ③ $Y = AB + BC + CD + DA$.

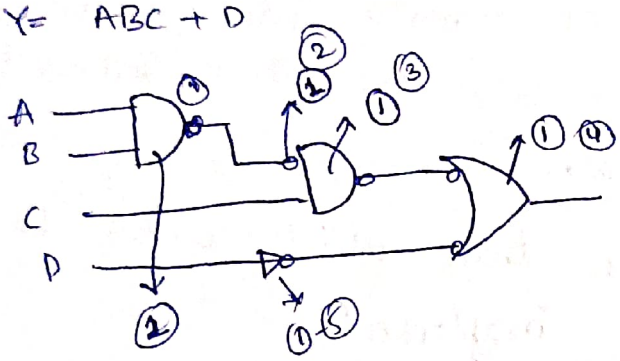
Soln $\Rightarrow B(A+C) + D(A+C)$



$\therefore$ Min Gate = ⑥

4 Qs. $Y = ABC + D$

Soln

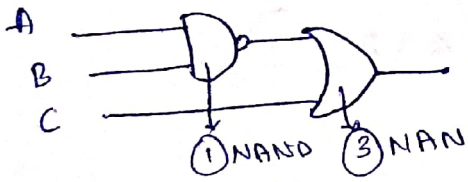


$\therefore$ Min Gate = ⑤

5Qs $Y = \bar{A} + \bar{B} + ABC$.

Soln $Y = \bar{A}B + (\overline{\bar{A}B})C$ $\{\because X + \bar{X}Y = X + Y\}$ (RLC)

$\therefore Y = \bar{A}B + C$.



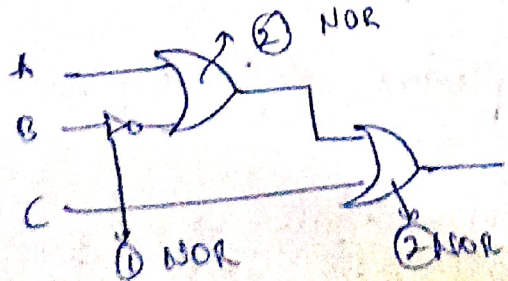
$\therefore$ Min Gate = ④

Case - ③ :-

- Minimum no of NOR Gate gate with single sum term.
- Do not minimize logic expression.
- Directly implement.

6Qs. $Y = A + \bar{B} + C$.

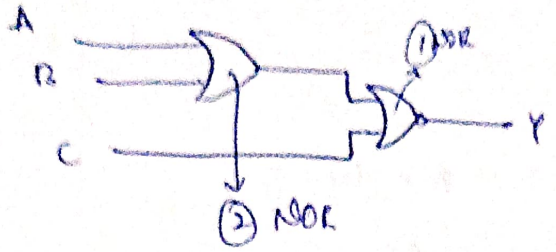
Sol.



Min Gate = ⑤

70) $Y = \overline{A+B+C}$

Soln

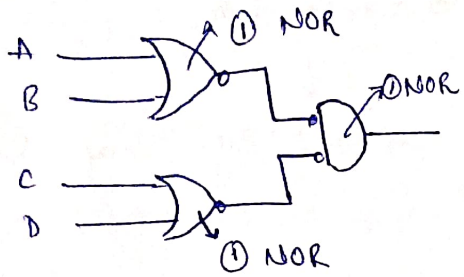


Min Gate = ③

* Case ④ :- Min no of NOR Gates when standard POS is given.
  - Min Expression.
  - Implement OR-AND.
  - Use concept of bubbling.

SQ) $Y = (A+B)\cdot(C+D)$.

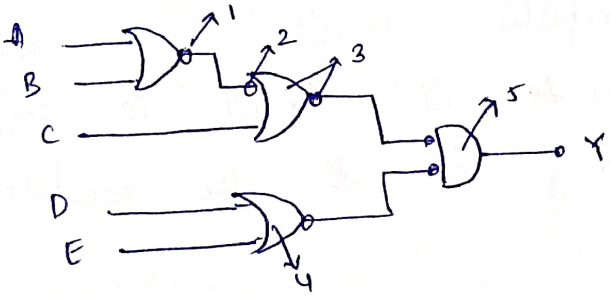
Sol



∴ Min Gate = ③.

SQ) $Y = (A+B+C)\cdot(D+E)$

Sol



∴ Min Gate = ⑤

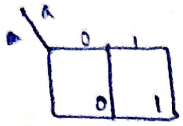
*** V.V.Imp :-

POS :- NOR
SOP :- NAND

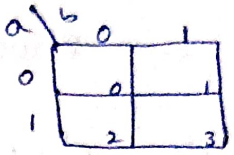
Karnaugh Map (K-Map).

- It is a graphical method, which consists of 2^n cells for 'n' variables. The adjacent cells have only one bit changes.
- It is suitable for 3 - 5-6 variables.

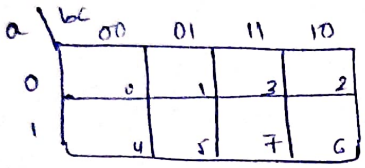
- One - variable :- $2^1 = 2$



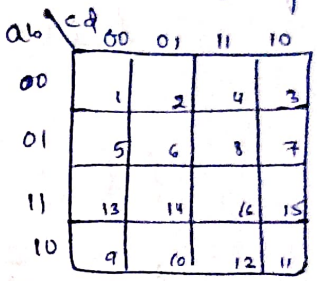
- 2- variable K-Map : $2^2 = 4$.



- 3- variable K-Map : $2^3 = 8$:



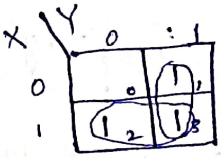
- 4 -variable K-Map :- $2^4 = 16$



1Q) Simplify the given 2 - variable Boolean eqn by using K-Map:

$$f = X\bar{Y} + \bar{X}Y + XY$$
$$10 \quad 01 \quad 11$$

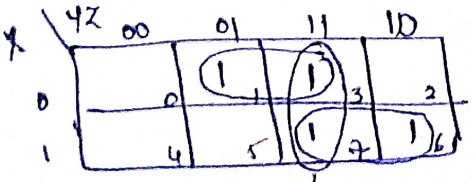
Soln $f = \Sigma m(1, 2, 3)$.



$\therefore$ $\boxed{F = X + Y}$

2Q) $$f = \bar{X}YZ + \bar{X}\bar{Y}Z + X\bar{Y}\bar{Z} + XYZ$$
$$011 \quad 001 \quad 110 \quad 111$$

$f = \Sigma m(3, 1, 6, 7)$



Redundant (No needed)

$f = (\bar{X} * Z) + XY$

$\therefore$ $\boxed{F = XY + \bar{X}Z.}$

20)

A \ BC	00	01	10	11
0	1	0	0	1
1	1	0	0	1

Soln Redraw to Normal form

A \ BC	00	01	11	10
0	1	0	1	0
1	1	0	1	0

$f \Rightarrow \overline{B}\overline{C} + B\overline{C}$

$f \Rightarrow \overline{B \oplus C}$

21) $f(A,B) = A' + B$. simplified Expression for $f(f(x+y, y), z)$ is

Soln $f(\overline{x+y} + y, z)$

$f \Rightarrow \overline{\overline{x+y} + y} + z$

$\Rightarrow \overline{\overline{x+y} * \overline{y}} + z$

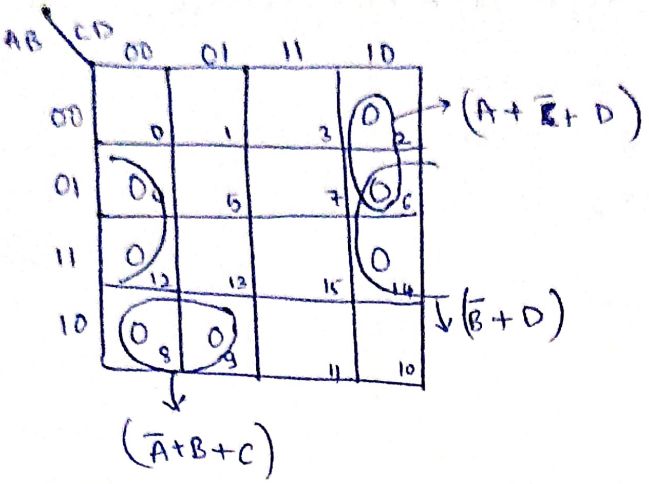
$\Rightarrow (\overline{\overline{x+y}}) \cdot \overline{y} + z$

$\Rightarrow xy + y\overline{y} + z$

$\therefore \boxed{f \Rightarrow xy + z}$

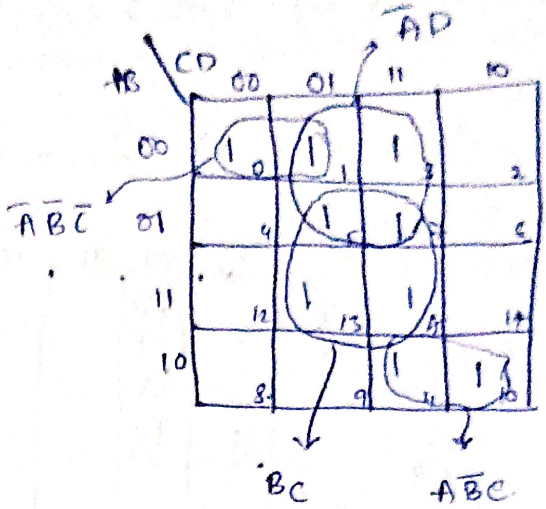
* 4- Variable K-Map :-

$f = \pi M(2,4,6,8,9,12,14)$



$f = (\bar{A}+B+C)\cdot(\bar{B}+D)\cdot(A+\bar{C}+D)$

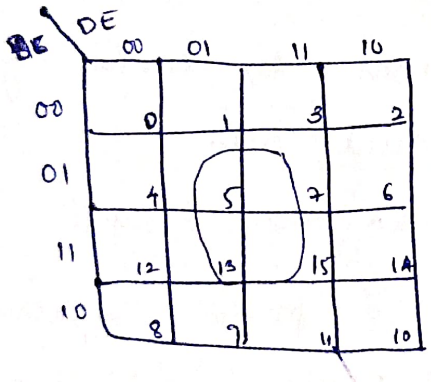
$f = \Sigma m(0,1,3,5,7,10,11,13,15)$



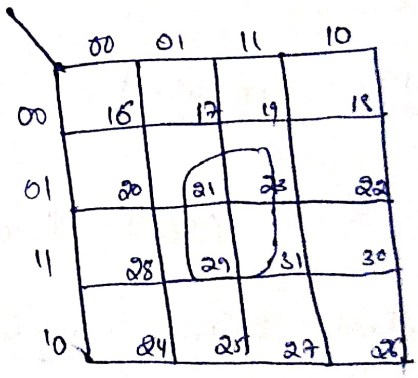
$f = \bar{A}\bar{B}\bar{C} + BD + A\bar{B}C + \bar{A}D$

* 5- variable K-Map :-

It have 32 minterms / 32 maxterms i.e 32 cells



A = 0

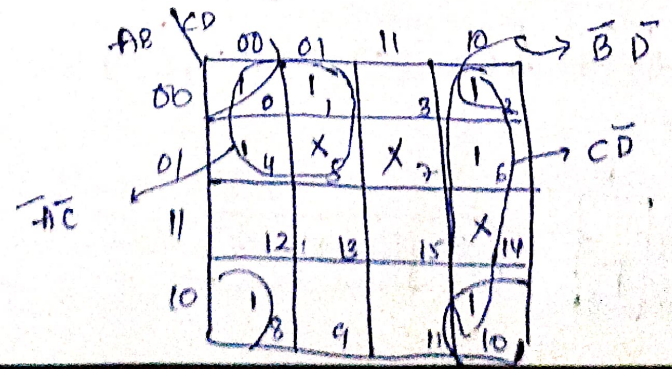


A = 1

* K-Map with "Don't Care" conditions :-

If $n \rightarrow$ no of don't care, then f_n will be 2^n.

$f(A,B,C,D) = \Sigma m(0,1,2,4,6,8,10) + d(5,7,14)$



$f = \bar{A}\bar{C} + \bar{C}\bar{D} + \bar{D}\bar{B}$

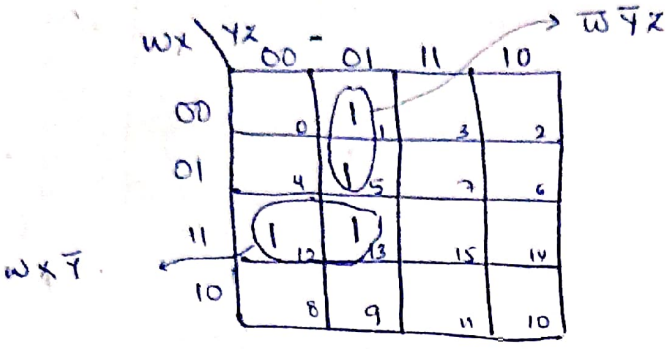
* Complement of a function :-

$f(A, B, C, D) = \Sigma m(0, 1, 3, 5, 8, 10, 12, 15) = \Pi M(2, 4, 6, 7, 9, 11, 13, 14)$

$\bar{f}(A, B, C, D) = \Sigma m(2, 4, 6, 7, 9, 11, 13, 14) = \Pi m(0, 1, 3, 5, 8, 10, 12, 15)$

Q). $F(W, X, Y, Z) = \Sigma(1, 5, 12, 13)$

Soln



$\therefore f = WX\bar{Y} + \bar{W}\bar{Y}Z.$

* K-Map Important Terminologies :-

(1). Implicant :- It is a nothing but min/max terms.

(2) Prime Implicant :- It is formed by combining maximum adjacent cells.

(Don't form smaller within larger groups).

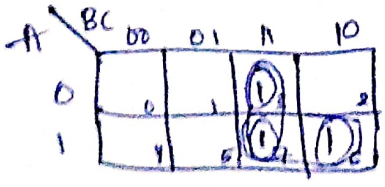
(3). Essential Prime Implicant :- It is a prime implicant which contains at least one min/max term, which cannot be covered by any other prime implicant.

Eg:- $f = AB + ABC + BC$

$\Rightarrow AB(C+\bar{C}) + ABC + (A+\bar{A})BC$

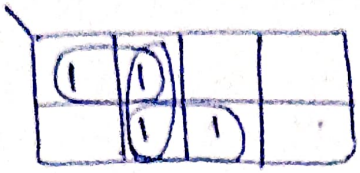
$\Rightarrow ABC + AB\bar{C} + ABC + ABC + \bar{A}BC$

$\Rightarrow \overset{111}{ABC} + \overset{110}{AB\bar{C}} + \overset{011}{\bar{A}BC}. \Rightarrow \Sigma m(7, 6, 3)$



$\therefore$ Implicant = 5

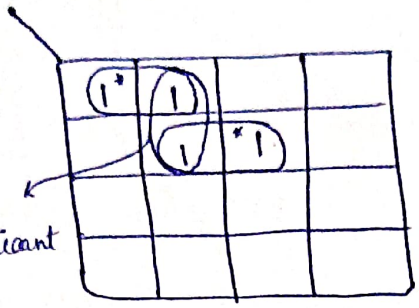
Example:-



form as many groups as possible & don't care about minz.

∴ Prime Implicant = (3)

Example:-



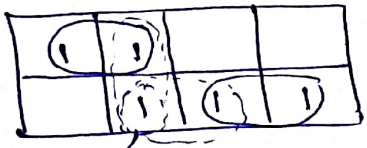
Redundant Prime Implicant (RPI)

↳ Both 1's are covered by some other groups.

∴ Esscatial Prime Implicants = (2)

Redundant Prime Implicants = (1).

*. Selective Prime Implicants :-



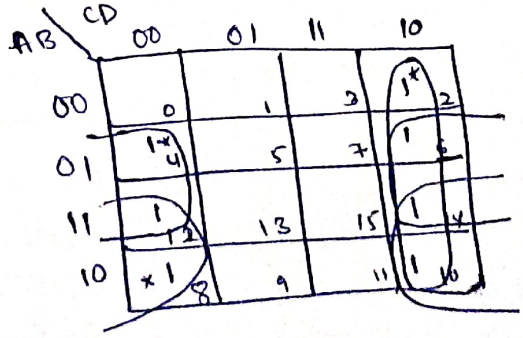
∴ Selective Prime Implicant = (2)

Only one will be present in minimized expression.

Q. Find I, PI, EPI, RPI & SPI ?

$f = \Sigma m(0, 4, 6, 8, 10, 12, 14)$.

Soln



Implicant = 15

PI = 3

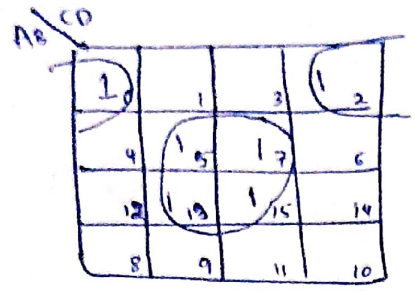
EPI = 3

RPI = 0

&PI = 0.

Q. $f = \Sigma m(0, 2, 5, 7, 13, 15)$

Sol:-



T = 12

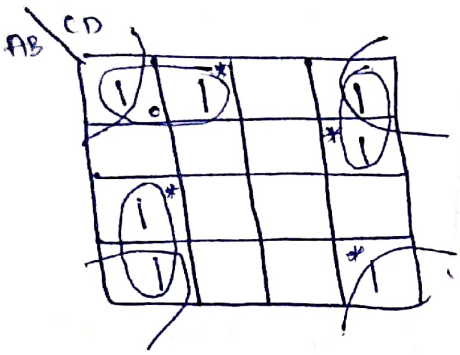
PI = 2

EPI = 2

RPI = 0

SPI = 0

Q. EPI = ?



EPI = (4)

Combinational Circuit.

- Combinational ckt consist of logic Gates & operates with binary values.
- The o/p depends on the combination of present i/p.

*. Combinational Circuits Design :-

Step 1 : Identify the problem statement.

Step 2 : Identify the no of i/p & o/p.

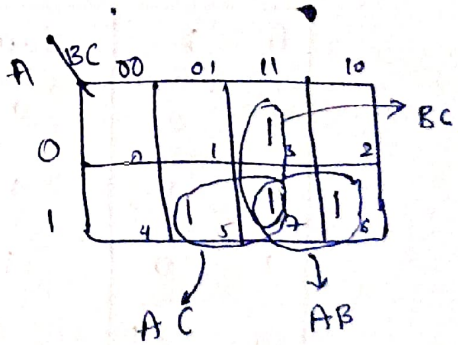
Step 3 : Determine SOP & POS for each o/p of ckt

Step 4 : Minimize SOP & POS expressions. (Using K-Map).

Step 5 : Draw logic Diagram.

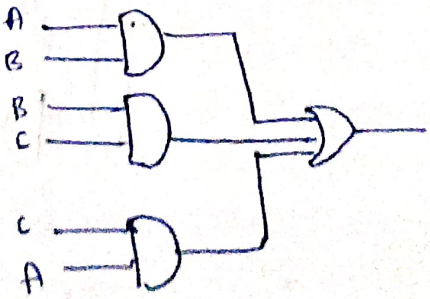
*. Majority Detector :- the ckt gives the o/p 1 when i/p have more no of 1's than 0's.

A	B	C	Y	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	m_3
1	0	0	0	
1	0	1	1	m_5
1	1	0	1	m_6
1	1	1	1	m_7



$Y = AB + BC + AC$.

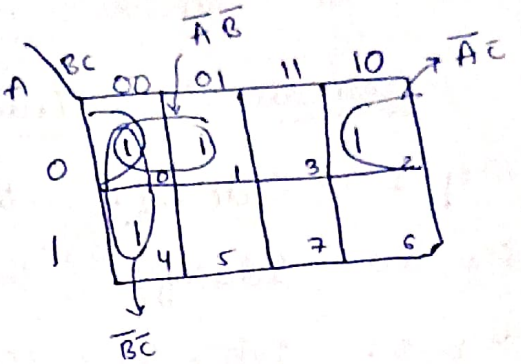
* Circuit Diagram :-



$Y = AB + BC + AC$.

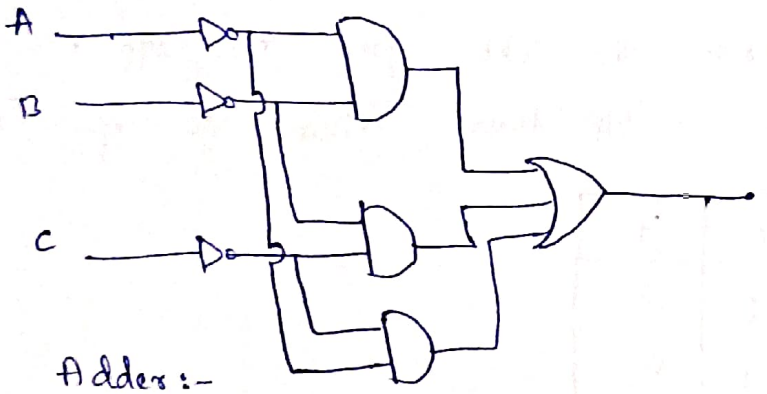
* Minority Detector :- This ckt gives the o/p 1 when i/p have lesser no of 1's than 0's.

A	B	C	Y	
0	0	0	1	m_0
0	0	1	1	m_1
0	1	0	1	m_2
0	1	1	0	
1	0	0	1	m_4
1	0	1	0	
1	1	0	0	
1	1	1	0	



$Y = \bar{A}\bar{B} + \bar{B}\bar{C} + \bar{C}\bar{A}$

Logic Diagram :-



$Y = \bar{A}\bar{B} + \bar{B}\bar{C} + \bar{C}\bar{A}$

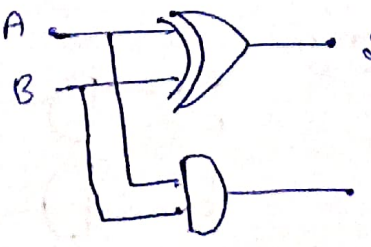
*. Half Adder :-

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

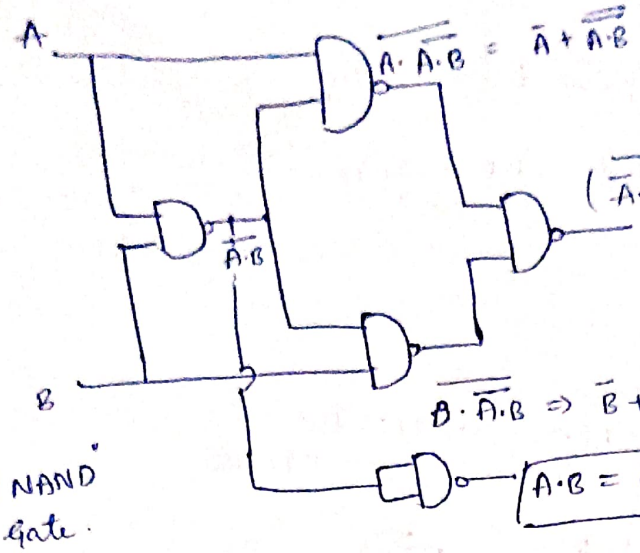
$Sum = A \oplus B$

$Carry = A \cdot B$

*. Circuit Diagram :-



Half Adder CKt Diagram Using NAND Gate & NOR Gate:-



$\Rightarrow (\overline{\bar{A} + AB}) + (\overline{\bar{B} + A \cdot B})$

$\Rightarrow (A \cdot \overline{AB}) + (B \cdot \overline{A \cdot B})$

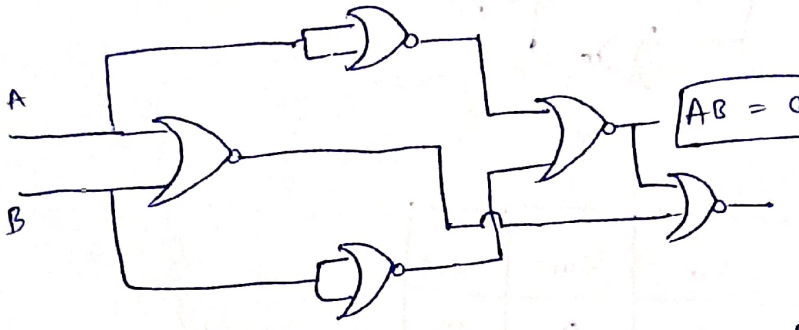
$= A \cdot (\bar{A} + \bar{B}) + B(\bar{A} + B)$

$\Rightarrow A\bar{A} + A\bar{B} + \bar{A}B + B\bar{B}$

$\Rightarrow A\bar{B} + \bar{A}B$

Sum => $A \oplus B$

(6) NAND Gate.



(5) NOR Gate.

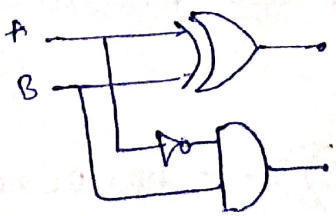
*. Half Subtractor :-

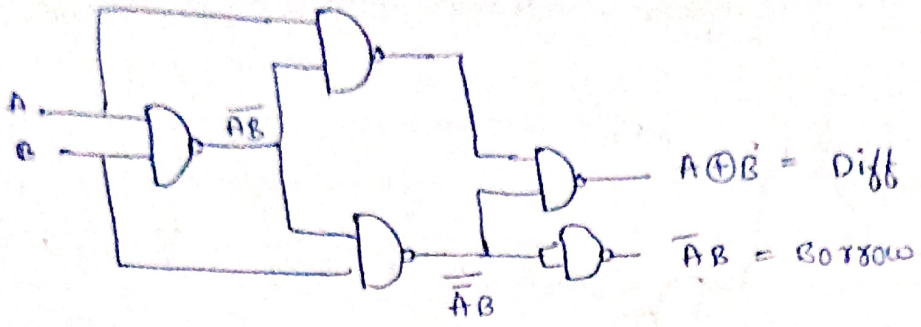
A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

∴ Diff = $A \oplus B$

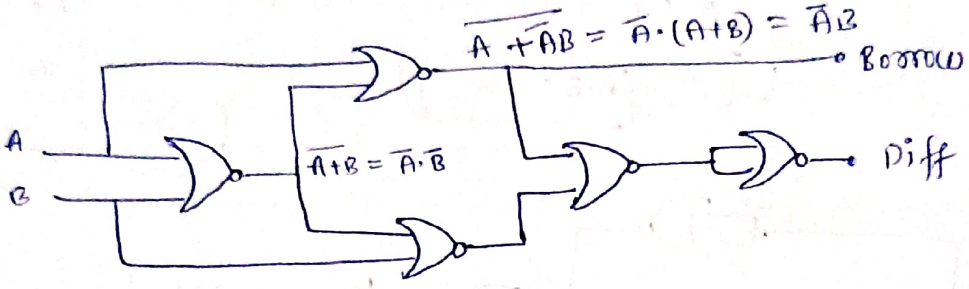
∴ Borrow = $\bar{A}B$

Circuit Diagram :-



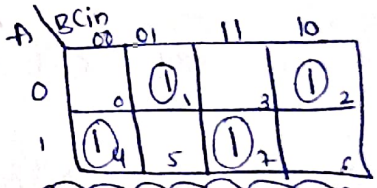


(5) - NAND Gate.

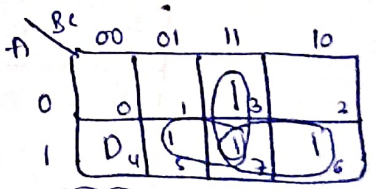


*. Full Adder :-

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

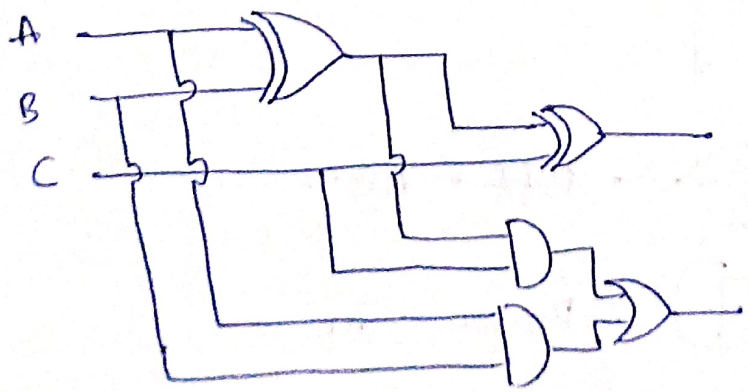


∴ Sum ⇒ $A \oplus B \oplus Cin$



∴ Carry = $AB + BC + CA$

*. Circuit Diagram :-

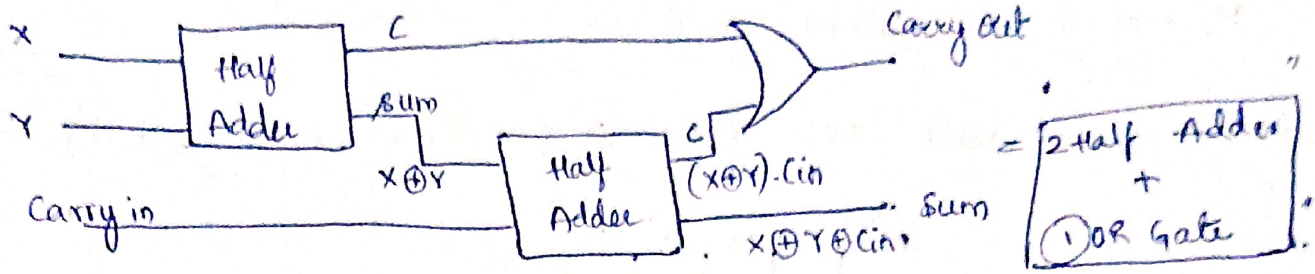


$A \oplus B \oplus Cin$ = Sum

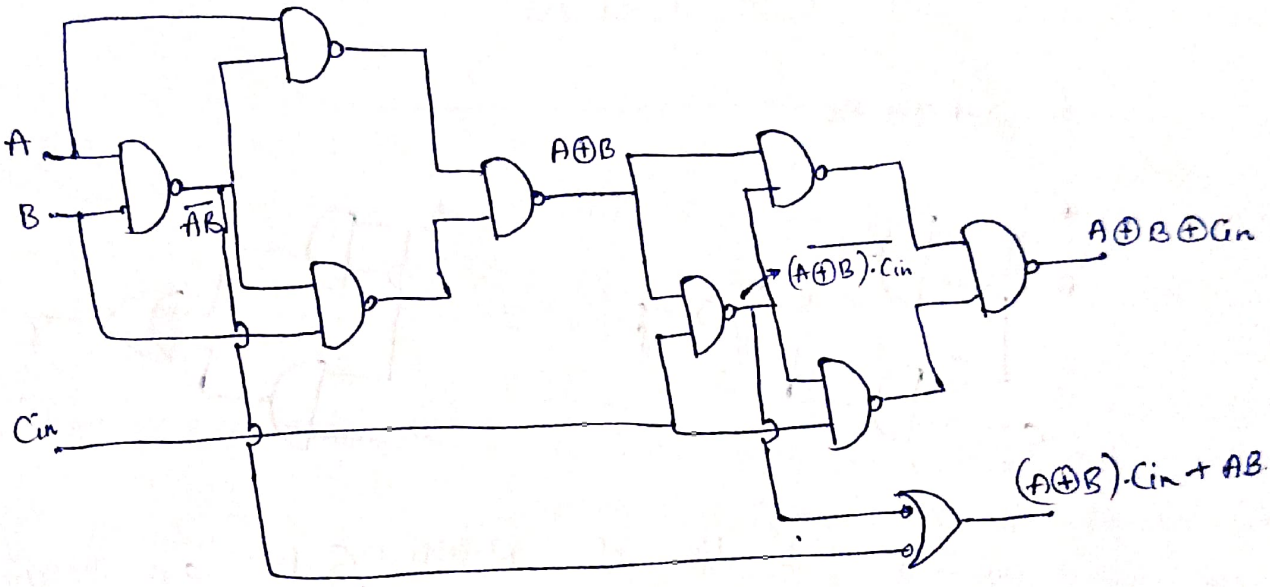
$AB + BC + CA$ = Carry.

$AB + (A \oplus B) \cdot Cin$

* Implementation of Full Adder Using Half Adder



*. Design of Full Adder Using NAND Gate :-



∴ No of NAND Gate Required to Implement NAND Gate based full adder is 9

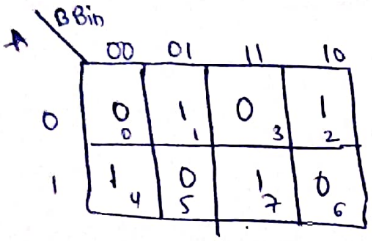
∴ "9" - NAND Gate

Similarly Implement Using NOR Gate

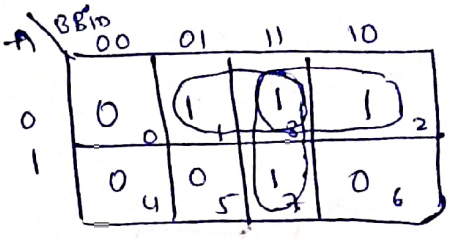
∴ "9" - NOR Gate

* Full subtractor :-

A	B	Borrow in	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



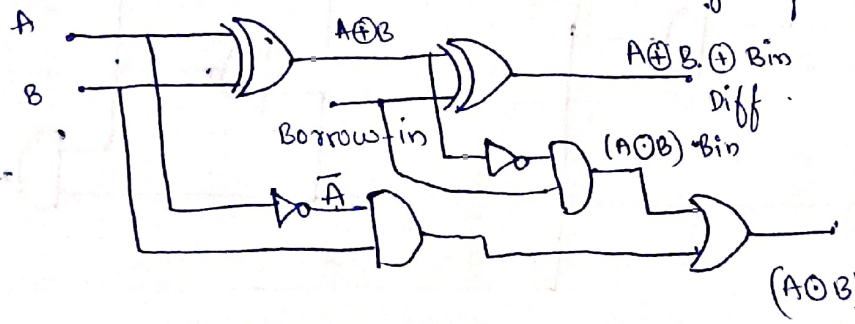
Diff = A ⊕ B ⊕ Bin.



Borrow ⇒ $\bar{A}$ Bin + $\bar{A}$ B + B Bin.

↳ Borrow can be obtained from Carry of full adder by taking comp. of minuend.

* Circuit Diagram :-



while converting Cout to Bout complement 'A'

∴ $\bar{A} \oplus B = A \odot B$

* Full Adder :-

∴ Sum ⇒ A ⊕ B ⊕ Cin.

Cout ⇒ AB + B Cin + A Cin

∴ Cout ⇒ (A⊕B) Cin + AB

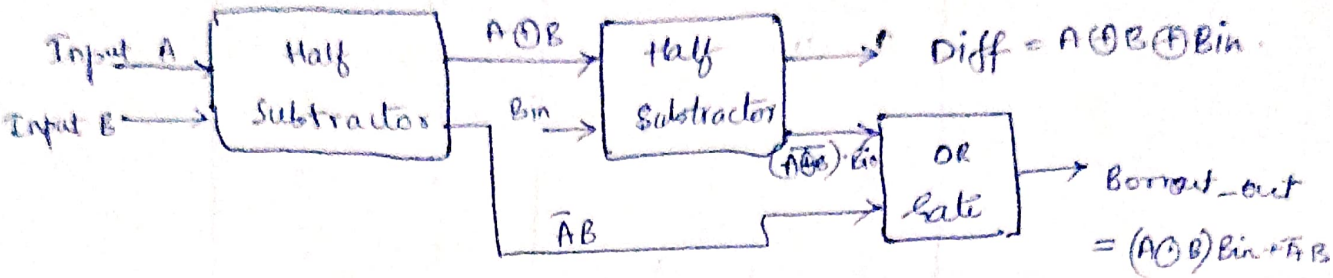
* Full subtractor :-

∴ Diff ⇒ A ⊕ B ⊕ Bin.

Borrow ⇒ $\bar{A}$B + $\bar{A}$ Bin + B Bin

∴ Borrow ⇒ (A⊙B) Bin + $\bar{A}$B

* Full subtractor Using Half Subtractor :-

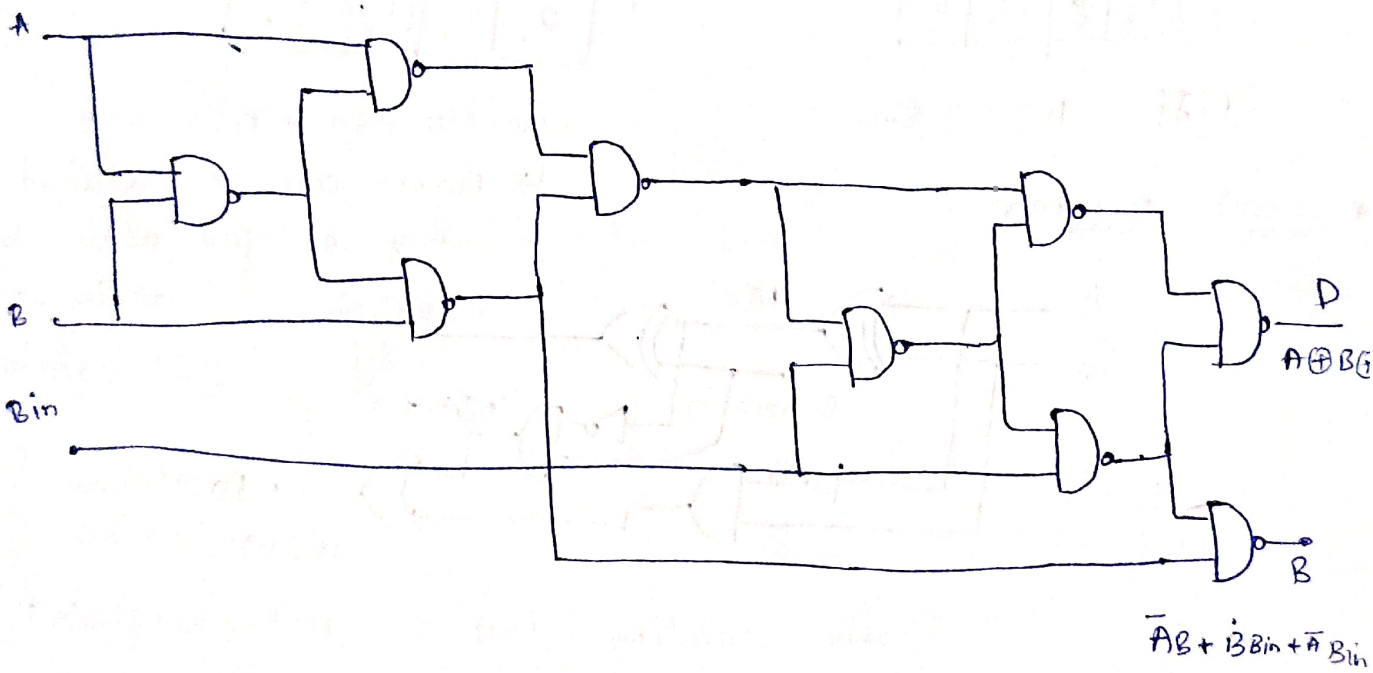


∴ Full subtractor = 2 Half Subtractor + One OR Gate.

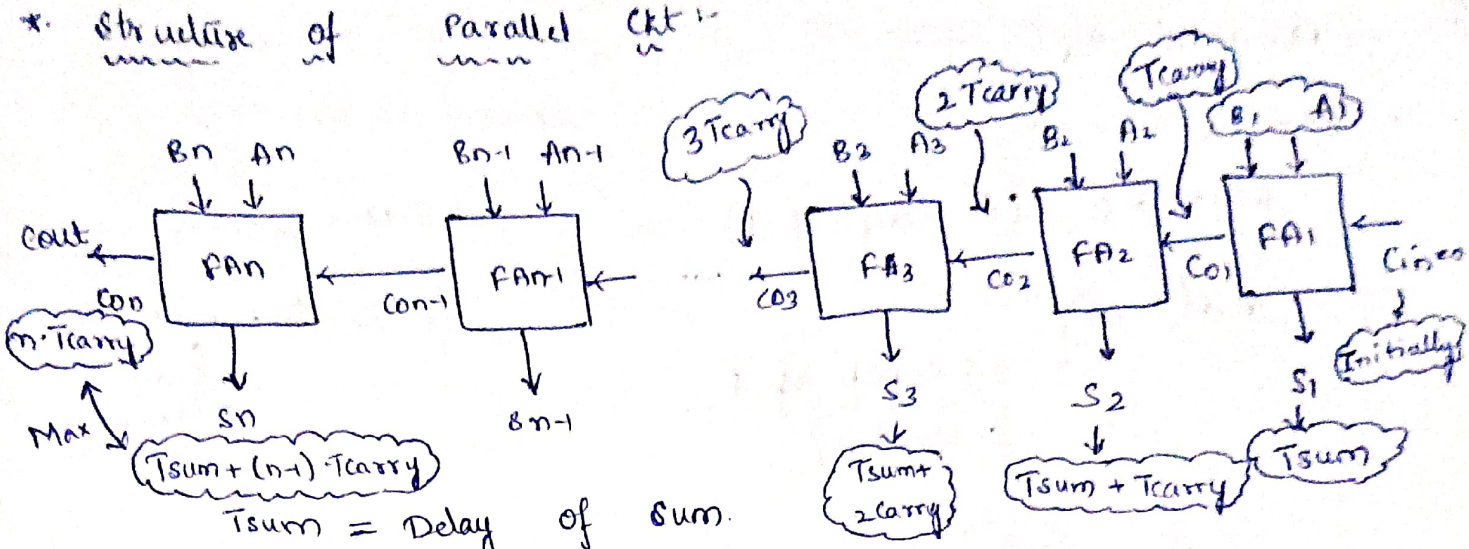
* Design of Full Subtractor using NAND Gate & NOR Gate

(1) NAND Gate

(2) NOR Gate



* Structure of Parallel Ckt :-



Tsum = Delay of Sum.

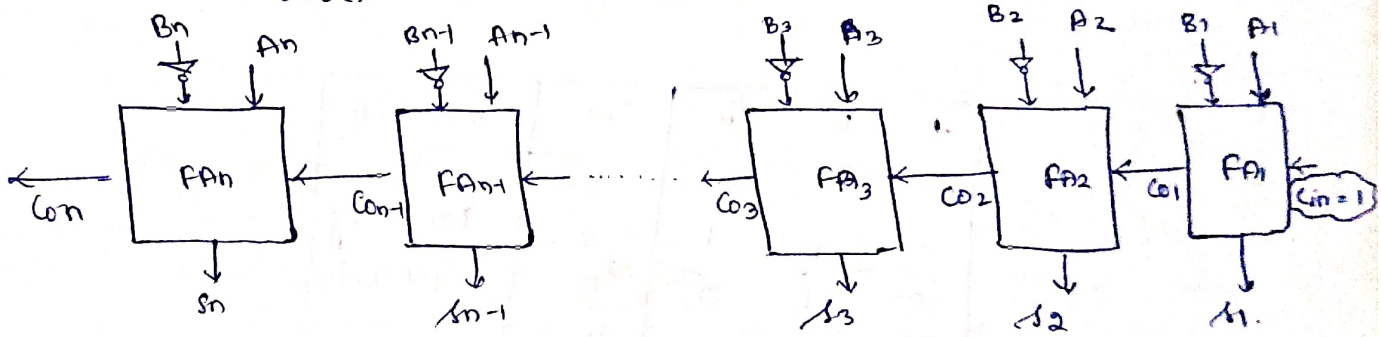
Tcarry = Delay of carry.

fig:- structure of Parrel Adder.

(or)

structure of Ripple Carry Adder.

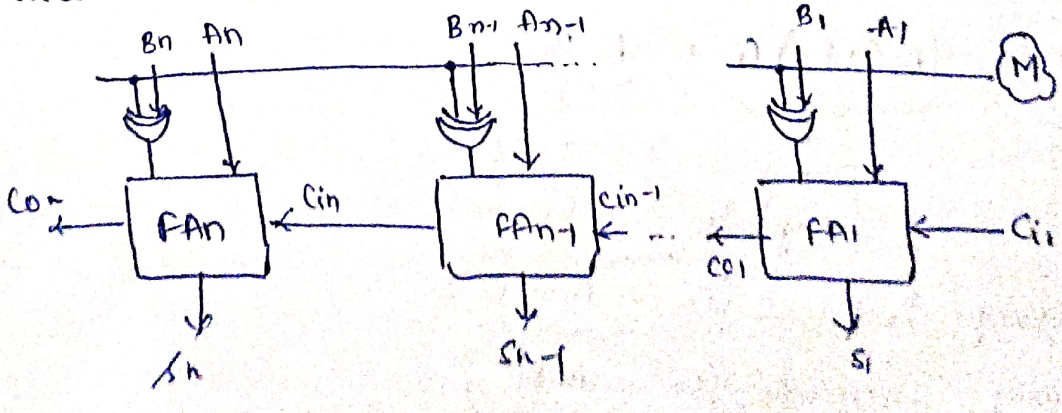
* Parallel Subtractor :-



A - B ⇒ A + 2's comp of B

⇒ A + (1's comp of B + 1)

* Parallel Adder / Subtractor :-



M = Mode I/p.

- M = 1, $C_i = 1$;
  $B \oplus M = \overline{B}$ (1's comp of B)
  $A + (\overline{B} + 1)$
  ↳ 2's comp of B
  (Subtractor)

- M = 0, $C_i = 0$.
  $B \oplus M = B$
  $A + (B + 0) = (A + B)$
  (Adder)

M = 0 (Adder)
M = 1 (Subtractor)

* Carry look - Ahead Adder :-

- By Using Ripple Carry Adder we are getting max delay at o/p.
- Hence By using carry look ahead adder, we ↓ delay.
  Cost of this adder is very high.

A	B	Cin	Cout	Condition
0	0	0	0	No Carry Generate
0	0	1	0	
0	1	0	0	
0	1	1	1	Carry Propagate
1	0	0	0	
1	0	1	1	
1	1	0	1	Carry Generate
1	1	1	1	

$$C_{i+1} = \underbrace{(A_i \oplus B_i) C_i}_{\text{Carry Propagate}} + \underbrace{A_i B_i}_{\text{Carry Generate}}$$

Carry Propagate $P_i = A_i \oplus B_i$
Carry Generate $G_i = A_i B_i$.

then sum & carry o/p can be expressed as :-

$$Si = Pi \oplus Ci$$

$$C_{i+1} = PiCi + Gi$$

for 4-bits :-

$C_1 = P_0 C_0 + G_0$; C_0 = i/p carry.

$C_2 = P_1 C_1 + G_1$

$= P_1 (P_0 C_0 + G_0) + G_1$

$= P_1 P_0 C_0 + P_1 G_0 + G_1$

$C_3 = P_2 C_2 + G_2$

$= P_2 P_1 P_0 C_0 + P_2 P_1 G_0 + P_2 G_1 + G_2$

By

$C_4 = P_3 P_2 P_1 P_0 C_0 + P_3 P_2 P_1 G_0 + P_3 P_2 G_1 + P_3 G_2 + G_3$.

Hence o/p depends on i/p carry & has less delay but ckt complexity ↑. Implement ckt diagram for the above expression.

★. Serial Adder :-